CLBIPP — Team Task Breakdown

v2 · wireframe-aligned

Stack: Next.js (TypeScript, App Router) · Prisma → Supabase Postgres · Supabase Auth · Supabase Realtime · Supabase Storage · Tailwind + shadcn/ui · Vercel

What changed in v2

- Wireframe is now the source of truth. Every screen maps to a section in clbipp_wireframe.html (or, for the Vendor Portal, the user-role description agreed on).
- "Vendor PWA" was a misnomer — that column is now correctly Field Agent App + Engine + AI.
- Vendor Portal (light viewing surface for suppliers) sits with Teammate 2's auth / components / PWA work.
- Entroview BMS sync is out. All BMS-style fields are manually entered in the Field Agent intake.
- Phased so dependencies are explicit. Don't start a phase task until the items it depends on are merged.

Roles

Person	Owns
You	Decision engine · Engine API surfaces · Field Agent App (intake → pickup → tracking) · QR system · AI features (Explain my price, MCP)
Teammate 1	DB schema & types · Admin Dashboard (all desktop screens) · Reference data (config, market prices) · Metals API
Teammate 2	Supabase Auth · Middleware & RLS · Shared component library · Vendor Portal · PDF certificates · Map view component · Deployment & PWA

Phase 0 — Setup (done)

- ☑ Supabase project created, env vars in GC
- ☑ Repo cloned by all three, `npm run dev` works
- ☑ Prisma migrated to Supabase
- ☑ `src/types/db.ts` exporting DB types
- ☑ Notion board live with this task list
- ☑ 30-min call locked the `/api/quote` contract, manual-input decision, and wireframe-fidelity goal

Phase 1 — Foundations (parallel, ~3–4 days)

These three streams run in parallel. The whole team unblocks once they land.

You

- ☑ `src/lib/decisionEngine.ts` — pure function, no DB/HTTP, 6-layer architecture
- ☑ `src/lib/decisionEngine.test.ts` — 20 hand-computed test cases via Vitest

Teammate 1

- ☐ Seed `config` table with working defaults (margin 25%, processing ₹50/kg, logistics ₹5/km/kg, refurb ₹180/kg, recovery efficiencies per metal)
- ☐ Seed `market_prices` table with placeholder values
- ☐ `GET /api/config` returns full config row
- ☐ `PATCH /api/config` updates parameters
- ☐ `GET /api/transactions` paginated + filterable
- ☐ `PATCH /api/market-prices` admin updates metal prices

Teammate 2

- ☐ Install `@supabase/ssr` and `@supabase/supabase-js`, create `src/lib/supabase.ts`
- ☐ Install shadcn/ui, configure `tailwind.config.ts`
- ☐ Build and export shared components: `Button`, `Input`, `Card`, `FormField`, `Badge`, `DataTable`, `Sidebar`, `PageLayout`, `Modal`, `Toast`
- ☐ `middleware.ts` at project root: intercepts `/api/*`, `/admin/*`, `/field/*`, checks Supabase session, attaches `user_id` + `role` to request headers, rejects unauth with 401
- ☐ `/login` page — email + password, Supabase Auth sign-in, redirects to role home
- ☐ `/signup` page — email + password + role selector (vendor / field_agent / admin)
- ☐ Supabase RLS policies: vendors read/write own `transactions` only; field agents read assigned transactions + write status updates; admins read all
- ☐ Connect repo to Vercel, all env vars set, confirm preview deploys on PRs

> ***Phase 1 ends when:*** *Teammate 1 has messaged GC that DB types are pushed and config / prices are seeded. Teammate 2 has messaged GC that middleware and components are merged. Nobody starts Phase 2 work that depends on these until then.*

Phase 2 — Core API + Engine wiring (~3 days)

The engine has to actually be reachable by the rest of the app.

You

- ☐ `POST /api/quote` — validates body against `QuoteInput` type, reads config + market prices via Prisma, calls `computeQuote()`, writes result + `vendor_id` (or `field_agent_id`) to `transactions` table, returns full `QuoteOutput`
- ☐ Auth guard on `/api/quote` once middleware is merged — saves `user_id` from session onto each transaction row
- ☐ `GET /api/qr/[transactionId]` — generates QR code (use `qrcode` npm package) encoding URL to public summary page

- ☐ `POST /api/explain/[transactionId]` — reads transaction, calls Anthropic API with breakdown JSON as context, returns plain-English explanation

Teammate 1

- ☐ `/admin/config` page — all engine parameters in editable fields, save button calls PATCH, success toast on save
- ☐ `/admin/prices` page — current metal prices editable with "Prices last updated [timestamp]" note
- ☐ Admin auth guards on `/admin/*` — non-admins redirected to `/login`

Teammate 2

- ☐ `POST /api/certificates/[transactionId]` — uses `@react-pdf/renderer`. Recycling cert: battery ID, weight, recovered materials, date, embedded QR. Refurb cert: battery ID, SoH reading, test pass result, date.
- ☐ `PATCH /api/transactions/[id]/status` — validates transition is legal (collected → tested → processed → certified, no skipping)
- ☐ PWA manifest: `manifest.json` with icons, theme colour, `display: standalone`. Configure `next-pwa`. Test install-to-home on Android Chrome.

> ***Phase 2 ends when:*** *Battery can be submitted via API → engine runs → transaction logged → QR generated → AI explanation works end-to-end (curl or Postman is fine).*

Phase 3 — Field Agent App: Intake & Pricing (~5–7 days)

The mobile flow from "agent arrived on site" to "price quoted to supplier." Build in order — each screen leads to the next.

You

- ☐ **Screen 02 · `/field/home`** — agent's day view: assigned pickups, quick stats (quotes today, average margin), "Scan / New Intake" CTA. Pulls `transactions` filtered by `field_agent_id` + status.
- ☐ **Screen 03 · `/field/scan`** — camera-based QR scanner via `html5-qrcode`. On scan, routes to existing transaction or new intake. Fallback: "Enter manually" → `/field/intake/new`.
- ☐ **Screen 04 · `/field/intake/new`** — manual battery entry form: battery type (LFP/NMC622/LCO/NCA), SoH %, capacity kWh, weight kg, age months, cycle count. Tooltips on each field (agents are reading off labels, not a BMS). Validation + sensible placeholders. Creates a draft transaction.
- ☐ **Screen 06 · `/field/intake/[id]/damage`** — 3-test damage rubric (Layer 1): visual inspection, swelling check, leak/burn marks. Each test scored 0–3 with photo upload for proof. Photos uploaded to Supabase Storage under `damage-evidence/{transactionId}/`. Weighted damage score computed live as tests are filled.
- ☐ **Screen 07 · `/field/intake/[id]/computing`** — animated loading screen ("Running Layer 0 · Intake validated... Layer 5 · Selection complete"). Calls `POST /api/quote` in background. ~1.5 second display.

- ☐ **Screen 08–10 · `/field/intake/[id]/result`** — three sections:
  - **Verdict** (Screen 08): pathway hero, P_min / P_recommended / P_max price band, net value.
  - **Breakdown** (Screen 09): revenue by metal (or resale), every cost line (logistics, processing, refining, storage, overhead), net profit, margin %.
  - **Why** (Screen 10): plain-English rationale, alternative pathways with deltas, sensitivity table, audit footer (config version, market snapshot, FX rate). "Why this price?" button → calls `/api/explain` → renders AI explanation below.

> ***Phase 3 ends when:*** *A field agent can scan or manually input a battery, complete damage scoring, see the engine output, and explain it to the supplier. The whole intake-to-quote flow works on a real Android phone.*

Phase 4 — Field Agent App: Pickup & Tracking (~5 days)

Once the quote is accepted, the battery has to actually be collected and tracked.

You

- ☐ **Screen 11 · `/field/intake/[id]/pickup`** — signature capture (`react-signature-canvas`), photos of battery + sealed packaging uploaded to Supabase Storage under `pickup-evidence/{transactionId}/`, scheduled drop-off datetime, supplier contact confirmation. On submit → `PATCH /api/transactions/[id]/status` to `collected`.
- ☐ **Screen 12 · `/field/intake/[id]/receipt`** — success screen: pickup ID, summary, "Share receipt" (calls cert API), "Back to home."
- ☐ **Screen 13 · `/field/pickups`** — tracking list. Agent's active pickups grouped by status. Each row tappable → opens timeline. Status badges, ETA hints.
- ☐ **Screen 14 · `/field/pickups/[id]`** — lifecycle timeline. Status events from `transaction_events` rendered as a vertical timeline: Collected → Transit → Warehouse → Refurb (current state pulses) → QA → Resold/Recycled. Each event shows timestamp, location, operator ID, notes. Subscribed via Supabase Realtime to `transaction_events` where `transaction_id = current` — status changes appear live without refresh. Component lives in `src/components/LifecycleTimeline.tsx` and is re-used by the Vendor Portal.
- ☐ **Screen 16 · `/field/history`** — agent's completed pickups, searchable + filterable. Columns: battery type, pathway, purchase price, date, status badge.

Teammate 2

- ☐ **Screen 15 · `/field/pickups/[id]/map`** — map view: pickup location and current battery location. Leaflet + OpenStreetMap (free, no API key). Markers for pickup, current position, destination. Component `src/components/BatteryMap.tsx` shared with admin traceability.
- ☐ **Screen 17 · `/field/profile`** — agent details, performance stats (pickups completed, avg margin), offline queue indicator, version, sign out.

> ***Phase 4 ends when:*** *Full field agent flow works on a phone: scan → intake → damage score → quote → pickup confirm → tracking timeline updates live as admin moves status forward.*

Phase 5 — Vendor Portal (~3–4 days)

The supplier's view. Uses shared components and the LifecycleTimeline component built in Phase 4.

Teammate 2

- ☐ `/vendor/home` — vendor dashboard: list of their batteries grouped by status, "Submit new" CTA, account stats.
- ☐ `/vendor/submit` — light self-service quote form. Same fields as `/field/intake/new` but vendor-facing (no damage scoring — that happens on physical inspection). Posts to `/api/quote` with `source: 'vendor_self'`; returns indicative quote with caveat.
- ☐ `/vendor/track/[id]` — vendor's view of their battery's lifecycle. Re-uses `LifecycleTimeline` component, also Realtime-subscribed. "Where is my battery" framing rather than operator detail.
- ☐ `/vendor/history` — past submissions, filterable, certificate download links once status is `certified`.
- ☐ `/vendor/certificates/[id]` — PDF cert viewer with download button.
- ☐ `/vendor/profile` — name, contact info, total batteries submitted, average margin received.

> ***Phase 5 ends when:*** *A vendor can sign up, submit a battery, see an indicative quote, watch the lifecycle update in real time as the field agent and admin move it through statuses, and download a certificate when done.*

Phase 6 — Admin Dashboard (~6–7 days)

The desktop side. Teammate 1 owns this phase end-to-end.

Teammate 1

- ☐ **Screen 18 · `/admin/overview`** — KPIs (quotes today, batteries in flight, avg margin, exception count), pathway split pie, margin trend line, market state panel, live queue preview (last 10 quotes). Recharts for all charts.
- ☐ **Screen 19 · `/admin/queue`** — quote queue table: filterable by status, pathway, supplier, agent. CSV export. Row click → battery detail.
- ☐ **Screen 20 · `/admin/battery/[id]`** — battery detail audit drilldown. Full `QuoteOutput` rendered: every input, every layer's output, audit metadata (config version, market snapshot, FX rate, input hash, engine version). Reproducible from logged inputs alone.
- ☐ **Screen 21 · `/admin/config`** — versioned engine config editor: damage weights, SoH gate thresholds, margin tiers, rate cards. Every save creates a new config version. Version history visible.
- ☐ **Screen 22 · `/admin/suppliers`** — suppliers directory. CRUD: name, tier (A/B/C), margin override, verified flag, contact. Each supplier links to their submission history.
- ☐ **Screen 23 · `/admin/traceability`** — network/fleet view. Map (`BatteryMap` component) with cluster pins per warehouse/refinery. Side panel showing flow volume between nodes.

- ☐ **Screen 24 · `/admin/exceptions`** — exception queue: transactions flagged HOLD or REVIEW. Each item shows reason (damage cap exceeded, unverified supplier, negative price), context, "Approve override" or "Reject" buttons. Action writes audit log entry.
- ☐ **Screen 25 · `/admin/compliance`** — CPCB/EPR reporting. Recovery yield by metal, period filters, PDF export of compliance summary (re-uses cert generation infrastructure).
- ☐ Attempt metals API integration (try metalpriceapi.com — check lithium and cobalt on free tier). If yes, replace manual price entry with scheduled daily fetch.
- ☐ Analytics polish: ₹ number formatting, loading skeletons, empty states.

> ***Phase 6 ends when:*** *Admin can monitor everything, configure anything, override exceptions, and pull compliance reports. The dashboard side feels operationally complete.*

Phase 7 — AI Features (~3 days)

You

- ☐ AI "Explain my price" polish — tune system prompt for plain-English non-technical tone (focuses on top revenue and cost drivers), cache responses on the transaction row so repeated clicks don't re-call Anthropic, add a "Regenerate" button.
- ☐ **MCP server (if time permits)** — Model Context Protocol server exposing the `transactions` table read-only. Admin running Claude Desktop can query in natural language ("Show me all NMC batteries processed in May with margin under 15%"). Implementation: `@modelcontextprotocol/sdk` Node server with one parameterised Prisma query tool.

Phase 8 — Integration, PWA, Polish (~4 days — all three)

All three

- ☐ End-to-end integration test: vendor signs up → submits battery → field agent picks up → agent moves through pickup flow → quote computed → QR scanned → admin sees transaction → admin updates status → vendor sees timeline update live → certificate generates → admin approves compliance entry.
- ☐ PWA finalisation (Teammate 2 leads): test on real iOS Safari (camera permissions behave differently), offline fallback page, confirm full vendor + field agent flow works in installed PWA mode.
- ☐ Record 5-minute demo video: vendor flow first, then field agent flow on mobile, then admin dashboard.
- ☐ Write `README.md`: setup instructions, architecture overview, env vars reference, team credits.
- ☐ Final bug sweep.
- ☐ Deploy to production on Vercel main domain.

Contracts

One person defines each, everyone else codes against it. Lock in Notion before deep coding — non-negotiable.

Contract	Owner	Status
DB TypeScript types (`src/types/db.ts`)	Teammate 1	Locked
Decision engine signature (`computeQuote()`)	You	Locked
`/api/quote` request/response shape	You + Teammate 1	Locked
Auth middleware contract (`user_id`, `role` headers)	Teammate 2	Locked
Shared component props	Teammate 2	Locked
Lifecycle timeline component props	You	New — lock before Phase 4
Status transition state machine	Teammate 2	New — lock before Phase 4
Map component props (`BatteryMap`)	Teammate 2	New — lock before Phase 4

Effort Balance (rough complexity units)

Person	P1	P2	P3	P4	P5	P6	P7	P8	Total
You	6 (done)	6	14	12	—	—	6	2	46
Teammate 1	8	4	—	—	—	18	—	2	32+
Teammate 2	16	6	—	5	11	—	—	6	44

Roughly balanced. Teammate 1's number is lower because admin dashboard work, while many screens, is mostly CRUD over the same database — patterns repeat. The data + metals API integration work makes up the difference.

Ground Rules

- No direct pushes to main — branch → PR → one review → merge
- Daily 10-min standup, voice call, no agenda
- Blocked > 2 hours → message GC immediately
- Finished a task others are waiting on → message GC explicitly. Don't assume commits are noticed.
- Contracts go in Notion before deep coding — non-negotiable
- No new features in the last week — that week is for fixing